



Debug Detective

Read, find the bug, fix it, explain

Name:

Date:

★ I can read two scripts, name which one is wrong and what kind of bug it is, and fix it.

Be the debug detective

You are the debug detective! Bit (green) and Pixel (purple) both start on the green flag and should MEET on number 3. Each script has TWO moves. One script is wrong. Read both carefully, run them, then name WHICH script is wrong, WHAT KIND of bug it is, and FIX it.

The goal: meet on 3



Bit's script

when green flag clicked

forward 1

forward 2

Pixel's script

when green flag clicked

back 1

back 3

1 Run both

Trace each two-move script. Bit (home 0): ____ Pixel (home 5): _____. Which sprite does NOT meet on the goal 3?

2 Name the bug

The wrong script went PAST the goal. Which move made it overshoot, and what kind of bug is it (a number too big, the wrong direction, or a missing block)?

3 Fix it and explain

Write the fixed move: back _____. After the fix, where does Pixel land? In one sentence, explain why your fix works.

Big idea

A debug detective reads carefully, runs the code, and compares it to the goal. The script that misses is the buggy one. Name the bug, change just what is wrong, and run again to check. Here Pixel's 'back 3' was too big — 'back 1' lands it right on 3.



Debug Detective

Quick facilitation notes & answer key

What this worksheet teaches

The capstone of the subject and of the whole K–G3 set: students read two two-move sets, find WHICH is wrong and WHAT KIND of bug it is, fix it, and explain. The goal is to meet on 3. No coding experience needed.

Before you start (no computer needed)

No computer needed — paper and a pencil. You read the prompts aloud. Optional: two counters on a number line 0 to 5 (green Bit, purple Pixel) so children can trace both sets of moves and see which one misses the goal.

How to lead it (about 20–25 minutes)

1. Show them (you do). Trace both two-move sets; one overshoots the goal.
2. Do one together. Exercise 1 — Bit meets 3; Pixel overshoots. Exercise 2 — the second move made it overshoot — a wrong-number bug.
3. Let them try. Students write the fixed move and explain why it works (Ex 3).

Answer key (these are the verified correct answers)

Exercise 1 — Bit: 0, forward 1 → 1, forward 2 → 3 (meets goal). Pixel: 5, back 1 → 4, back 3 → 1 (overshoots; Pixel is buggy).

Exercise 2 — the second move "back 3" made Pixel overshoot. It is a wrong-number bug (the number is too big), not a direction or missing-block bug.

Exercise 3 — fixed move back 1 → Pixel: 5 → 4 → 3. From 4, the goal 3 is one square back, so back 1 lands exactly on 3. The direction was right; only the size was wrong.

If students get stuck — and ways to adjust

Watch for: changing the direction. Here the direction (back) is right; only the size is too big.

Make it easier: trace Pixel's two moves and mark where it passes the goal.

Make it harder: ask them to name all four bug types they have learned (wrong number, wrong direction, missing block, mismatched message).

Ontario curriculum link (official wording)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential and concurrent events.

In plain terms: students write and run instructions, including two sets that run at the same time.

C3.2 — read and alter existing code, including code that involves sequential and concurrent events, and describe how changes to the code affect the outcomes.

In plain terms: students read two sets of instructions, find and fix the wrong one, and explain the fix.

You'll know it worked when

The child finds the buggy set, names the bug type, fixes it, and explains why the fix lands on 3.